UD6.2 Detección y corrección de errores (anexo)

Introducción

Cuando se transmiten bits por un medio físico (cable de cobre, fibra, aire…) pueden alterarse a causa del ruido, las interferencias o la atenuación de la señal: un 0 llega como un 1, o al revés. Entre los servicios de la capa de enlace que vimos en la UD6.1 está precisamente el control de errores.

La idea común a todas las técnicas es añadir a los datos unos bits redundantes, calculados a partir de ellos, que permiten al receptor comprobar si la información ha llegado bien. Hay dos objetivos posibles:

- **Detección de errores:** el receptor sabe que ha habido un error, pero no cuál. Normalmente descarta la trama y se retransmite (la retransmisión la gestiona el propio protocolo de enlace o, más habitual en Ethernet, un protocolo de capas superiores).
- **Corrección de errores:** el receptor es capaz de averiguar qué bit está mal y corregirlo sin necesidad de retransmitir.

En este anexo veremos tres técnicas:

- **Bit de paridad:** la más sencilla; detecta algunos errores.
- **CRC (Cyclic Redundancy Check):** detecta casi cualquier error; es la que usan Ethernet y Wi-Fi.
- **Código de Hamming:** permite corregir errores; solo veremos que existe.

¿Qué campo de la trama Ethernet se utiliza para detectar errores de transmisión?

Bit de paridad

Idea

Consiste en añadir un único bit al final de un bloque de datos para que el número total de unos (incluido ese bit) tenga una paridad determinada:

- **Paridad par:** el número total de unos debe ser par.
- **Paridad impar:** el número total de unos debe ser impar.

Emisor y receptor deben haberse puesto de acuerdo previamente en qué tipo de paridad usan.

Cómo se calcula

1. Se cuenta cuántos unos hay en los bits de datos.
2. Con **paridad par**: si el número de unos es par, el bit de paridad es 0; si es impar, es 1 (para "completar" hasta un total par).
3. Con **paridad impar**: si el número de unos es par, el bit de paridad es 1; si es impar, es 0 (para "completar" hasta un total impar).
4. Se añade el bit de paridad al final de los datos y se transmite todo junto.

Ejemplo con caracteres ASCII de 7 bits (el octavo bit del byte se usa como bit de paridad):

Carácter	Dato (7 bits)	Nº de unos	Bit paridad par	Bit paridad impar	Byte enviado (par)
A	1000001	2	0	1	10000010
C	1000011	3	1	0	10000111
a	1100001	3	1	0	11000011
z	1111010	5	1	0	11110101
7	0110111	5	1	0	01101111

Comprobación en el receptor

El receptor cuenta los unos de todos los bits recibidos (datos + bit de paridad). Si cumple la paridad acordada, da la trama por buena; si no, sabe que ha habido un error.

```
Paridad par. Recibido: 10110101
  N° de unos = 5  (impar)  ->  ERROR

Paridad par. Recibido: 01101001
  N° de unos = 4  (par)    ->  correcto
```

Limitaciones

- **Solo detecta un número impar de bits erróneos** (1, 3, 5…). Si se alteran 2 bits, el número de unos sigue teniendo la misma paridad y el error pasa desapercibido.
- **No localiza el error:** sabemos que hay un fallo, pero no en qué bit.
- **No corrige:** solo detecta, por lo que hay que retransmitir.

> **Importante:** que la comprobación de paridad dé "correcto" no garantiza que los datos sean correctos; solo que no se ha detectado ningún error.

Paridad bidimensional (ampliación)

Si organizamos los datos en filas y columnas y calculamos un bit de paridad para cada fila y para cada columna, un error en un solo bit hace fallar una fila y una columna a la vez. Su intersección indica exactamente qué bit es el erróneo, así que en este caso sí se puede corregir. Ejemplo con paridad par:

```
Enviado:                  Recibido (error en fila 2, col. 3):
  1 0 1 1 | 1               1 0 1 1 | 1   fila 1: OK
  0 1 1 0 | 0               0 1 0 0 | 0   fila 2: FALLA (1 uno)
  1 1 0 0 | 0               1 1 0 0 | 0   fila 3: OK
  --------+--               --------+--
  0 0 0 1 | 1               0 0 1 1 | 1
                                ^
                            columna 3: FALLA
```

Esta idea, usar varios bits de paridad que se solapan para poder localizar el error, es la base del código de Hamming.

CRC (Cyclic Redundancy Check)

Idea

El CRC es una técnica de detección de errores mucho más potente que la paridad. El emisor y el receptor se ponen de acuerdo en un número binario llamado polinomio generador. El emisor divide los datos entre ese polinomio y el resto de la división (el FCS, Frame Check Sequence, ya visto en la trama Ethernet) se añade al final de la trama. El receptor repite la división con la trama completa:

- **Resto = 0:** la trama se da por correcta.
- **Resto distinto de 0:** hay un error, y la trama se descarta.

Aritmética módulo 2

Las divisiones del CRC se hacen en módulo 2: no hay acarreos y tanto la suma como la resta equivalen a la operación XOR (o exclusivo):

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

El polinomio generador y su representación binaria

El polinomio generador se escribe como un polinomio, pero para calcular lo pasamos a binario: cada coeficiente (1 si el término está presente, 0 si no) es un bit, empezando por la potencia más alta. Por ejemplo, $P(x) = x^3 + x + 1$ es $1 \cdot x^3 + 0 \cdot x^2 + 1 \cdot x + 1$, es decir, 1011.

Polinomio P(x)	Divisor (binario)	Bits del FCS (grado)
$x^3 + x + 1$	1011	3
$x^2 + 1$	101	2
$x^4 + x^2 + 1$	10101	4
$x + 1$	11	1

> **Regla:** el divisor tiene siempre (grado + 1) bits, y el FCS resultante tiene tantos bits como el grado del polinomio. Fíjate en el último caso: el CRC con x + 1 es exactamente un bit de paridad par.

Cálculo del FCS en el emisor

1. Pasar el polinomio generador a binario. Sea r su grado (número de bits del FCS).
2. Añadir r ceros al final de los datos.
3. Dividir en módulo 2 el resultado entre el divisor. Mientras el bit de más a la izquierda del tramo actual sea 1 se hace XOR con el divisor; si es 0, se desplaza un bit.

4. El resto, escrito con exactamente r bits (rellenando con ceros por la izquierda si hace falta), es el FCS.
5. La trama que se envía es: datos + FCS.

Ejemplo: datos = 100101, $P(x) = x^3 + x + 1$ (divisor 1011, r = 3). Se añaden 3 ceros a los datos y se divide 100101000 entre 1011:

```
100101000
1011
---------
001001000
  1011
---------
000010000
    1011
---------
000000110
```

El resto es **110**, así que FCS = **110** y la trama enviada es **100101110**.

Comprobación en el receptor

El receptor divide la trama recibida entre el mismo divisor. Sin errores, T = 100101110:

```
100101110
1011
---------
001001110
  1011
---------
000010110
    1011
---------
000000000
```

Resto = **000** (todo ceros) -> la trama es correcta.

Ahora supongamos que el tercer bit se altera durante la transmisión y llega 101101110:

```
101101110
1011
---------
000001110
     1011
---------
000000101
```

Resto = **101** (distinto de cero) -> error detectado, la trama se descarta.

> **Observa:** el receptor no sabe qué bit falló; solo que el resto no es cero. El CRC detecta errores, no los corrige.

El CRC en Ethernet

El campo FCS de la trama Ethernet (UD6.1) contiene un CRC de 32 bits calculado por la tarjeta de red (por hardware) sobre la cabecera y los datos de la trama. El polinomio generador es el estándar CRC-32 de IEEE 802.3: $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x$

+ 1. Si el receptor detecta que el FCS no coincide, descarta la trama sin avisar; la recuperación (retransmisión) se deja a las capas superiores, como TCP.

Un CRC de r bits detecta siempre las ráfagas de errores de hasta r bits y, en la práctica, casi cualquier otro error. Eso sí, está pensado para errores accidentales: no protege frente a alguien que modifique la trama a propósito y recalcule el CRC.

Código de Hamming (solo conocerlo)

El código de Hamming, propuesto por Richard Hamming en los laboratorios Bell hacia 1950, es una técnica de corrección de errores. En lugar de un único bit de paridad, añade varios bits de paridad colocados en las posiciones 1, 2, 4, 8… de la palabra, y cada uno vigila un subconjunto distinto de bits. Al comprobarlos, el patrón de bits de paridad que fallan indica la posición exacta del bit erróneo, de modo que el receptor puede darle la vuelta y corregirlo.

- Corrige errores de un solo bit (y detecta algunos de dos).
- Tiene un coste: añade más bits redundantes que la paridad o el CRC. Por ejemplo, Hamming(7,4) usa 3 bits de paridad por cada 4 de datos.
- Se usa donde retransmitir es difícil o caro, como en memorias RAM con ECC o comunicaciones espaciales. En Ethernet no se usa: allí es más eficiente detectar con CRC y retransmitir.

No vamos a estudiar cómo se calcula; basta con saber que existe y para qué sirve.

Resumen comparativo

Técnica	Bits añadidos	Detecta	Corrige	Ejemplo de uso
Paridad simple	1	Nº impar de bits erróneos	No	Comunicaciones serie sencillas
Paridad 2D	1 por fila y 1 por columna	Más errores que la paridad simple	1 bit	Ampliación didáctica
CRC	r (p. ej. 32)	Casi todos los errores	No	Ethernet, Wi-Fi
Hamming	Varios (3 por cada 4 de datos)	Errores de 1 y 2 bits	1 bit	Memoria RAM ECC

Ejercicios

Bit de paridad

P1. Calcula el bit de paridad **par** de los siguientes datos y escribe el byte que se enviaría (datos + bit de paridad).

	Dato	Nº de unos	Bit de paridad	Byte enviado
a)	1011001			
b)	0110000			
c)	1111111			
d)	0000001			

P2. Igual que el ejercicio anterior, pero con paridad **impar**.

	Dato	Nº de unos	Bit de paridad	Byte enviado
a)	1100110			
b)	0101011			
c)	1000000			
d)	1111110			

P3. Actúa como receptor. Se usa paridad par y el último bit de cada byte es el bit de paridad. Indica si cada byte recibido es correcto o contiene un error. ¿Podrías decir qué bit ha fallado en los que tienen error?

	Byte recibido	Nº de unos	¿Correcto o error?
a)	10110101		
b)	01101001		
c)	11100110		
d)	00111001		

P4. El emisor envía el byte 01101001 con paridad par. Para cada una de las situaciones, indica si el receptor detectará el error y explica por qué:

- a) Llega 01101001 (sin errores).
- b) Llega 01101011.
- c) Llega 01100011.

¿Qué conclusión sacas sobre las limitaciones del bit de paridad?

CRC

Ejercicios 1 a 4. Se utiliza el polinomio generador para calcular el FCS. ¿Cuál es la representación binaria del divisor y cuántos bits tendrá el FCS resultante?

Nº	Polinomio P(x)	Divisor (binario)	Bits del FCS
1	$P(x) = x^3 + 1$		
2	$P(x) = x^3 + x^2 + 1$		
3	$P(x) = x^4 + x + 1$		
4	$P(x) = x^2 + x + 1$		

Ejercicios 5 a 8. En los siguientes ejercicios, actúa como receptor. Se ha recibido una trama. Debes realizar la división módulo 2 de la trama recibida entre el polinomio generador y analizar el resultado.

Ejercicio	Trama recibida (T')	Polinomio P(x)	Resto	¿Correcta?
5	1101010	$P(x) = x^3 + x + 1$		
6	1011101	$P(x) = x^3 + x^2 + 1$		
7	1001110	$P(x) = x^3 + x + 1$		
8	1110011	$P(x) = x^3 + x^2 + 1$		

Bibliografía

- https://es.wikipedia.org/wiki/Bit_de_paridad
- https://es.wikipedia.org/wiki/Comprobaci%C3%B3n_de_redundancia_c%C3%ADclica
- https://es.wikipedia.org/wiki/C%C3%B3digo_Hamming